Module Guide for Software Engineering

Team #18, Gouda Engineers
Aidan Goodyer
Jeremy Orr
Leo Vugert
Nathan Perry
Tim Pokanai

April 10, 2026

1 Revision History

Date	Version	Notes
Nov 13, 2025	1.0	Initial Module Guide (Rev 0)
Apr 6, 2026	2.0	Final Revision: timeline overhaul, cross-document references, UI descriptions, margin compliance

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

Symbol / Abbreviation	Description
AC	Anticipated Change
API	Application Programming Interface
APP	Appearance Requirement
CAP	Capacity Requirement
DPP	Data Processing Pipeline (Module)
DSS	Data Schema and Storage (Module)
DV	Data Visualization (Module)
EOU	Ease of Use Requirement
EPE	Expected Physical Environment Requirement
FEM	Fault and Error Management (Module)
FR	Functional Requirement
FRDR	Federated Research Data Repository
FUNC	Functional Requirement (Prefix)
HH	Hardware-Hiding (Module)
HTTPS	Hypertext Transfer Protocol Secure
IMM	Immunity Requirement
INT	Integrity Requirement
IWAS	Interfacing with Adjacent Systems Requirement
JSON	JavaScript Object Notation
LEA	Learning Requirement
LEG	Legal Requirement
LON	Longevity Requirement
LLM	Large Language Model
M	Module
MAI	Maintenance Requirement
MG	Module Guide
NLP	Natural Language Processing
NLPQP	NLP Query Processor (Module)
NFR	Non-Functional Requirement
OCD	Obsessive-Compulsive Disorder

Symbol / Abbreviation	Description
OI	Open Issue
OS	Operating System
POA	Precision or Accuracy Requirement
PROD	Productization Requirement
R	Requirement
REL	Release Requirement
REST	Representational State Transfer
ROFT	Robustness or Fault Tolerance Requirement
SAL	Speed and Latency Requirement
SOE	Scalability or Extensibility Requirement
SRS	Software Requirements Specification
SUP	Supportability Requirement
UAP	Understandability and Politeness Requirement
UC	Unlikely Change
UD	User Documentation Requirement
UI	Front-End Interface (Module)
WE	Wider Environment Requirement
WR	Waiting Room (Deferred Requirement)
AAC	Authentication and Access Control (Module)
ADA	Adaptability Requirement
ACC	Accessibility Requirement
JSON API	Structured data exchange format over HTTP(S)

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms ii

3 Introduction **1**
3.1 Related Documents 1
3.2 Document Organization 2

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes 2
4.2 Unlikely Changes 3

5 Module Hierarchy **3**

6 Connection Between Requirements and Design **4**

7 Module Decomposition **4**
7.1 Hardware-Hiding Module (M1) 5
7.2 Behaviour-Hiding Modules 5
7.2.1 Front-End Interface Module (M3) 5
7.2.2 API Layer Module (M4) 5
7.2.3 NLP Query Processor Module (M2) 5
7.2.4 Authentication and Access Control Module (M8) 5
7.3 Software Decision Modules 6
7.3.1 Data Processing Pipeline Module (M7) 6
7.3.2 Data Schema and Storage Module (M5) 6
7.3.3 Data Visualization Module (M6) 6
7.3.4 Fault and Error Management Module (M9) 6

8 Traceability Matrix **6**

9 Use Hierarchy Between Modules **8**

10 User Interfaces **9**
10.1 Homepage 9
10.2 Query Page 12
10.3 Experiment and About Pages 14
10.4 Error Handling 16

11 Design of Communication Protocols **17**
11.1 Overall Communication Architecture 17
11.2 External Communication 17
11.3 Internal Communication 18
11.4 Communication Reliability and Standards 18

11.5 Scalability and Fault Tolerance 18

12 Timeline **18**

List of Tables

2 Module Hierarchy 4
3 Trace Between Requirements and Modules 6
4 Trace Between Anticipated Changes and Modules 8
5 Detailed Project Timeline and Responsibilities 18

List of Figures

1 Use hierarchy among modules 9
2 InfraRat Homepage – Initial Layout Frame 1. Displays the main navigation bar, branding, and hero section introducing the platform and its purpose to researchers. 10
3 InfraRat Homepage – Initial Layout Frame 2. Continues the landing page content, highlighting key features of the platform such as dataset querying and visualization capabilities. 10
4 InfraRat Homepage – Trial Data Display. Shows a summary view of trial data records retrieved from the Data Schema and Storage Module (M5), giving users a preview of the structured experimental session data. 11
5 InfraRat Homepage – NLP Information. Presents information about the natural language query capability powered by the NLP Query Processor Module (M2), explaining how researchers can use conversational input to search the dataset. 12
6 Query Page – Search and Filter Interface. Displays the dual-mode query interface where users can either apply structured metadata filters (left panel) or enter natural language queries (central input). This page exercises the Front-End Interface Module (M3), API Layer Module (M4), and NLP Query Processor Module (M2). 13
7 Query Page – Test of Sample Prompt. Demonstrates a completed query with results displayed in a tabular format returned from the Data Schema and Storage Module (M5) via the API Layer (M4). Users can view, sort, and download the returned dataset records. 14
8 Experiment Page – Experiment Management and Overview. Allows users to browse and manage experimental groupings, linking sessions through the Data Schema and Storage Module (M5). 15
9 About Page – Team Section. Presents team member information and the project's academic context. 15
10 About Page – Frequently Asked Questions (FAQ) and Background Information. Provides users with contextual information about OCD research, the FRDR dataset, and instructions for using the platform. 16

11 Error Page – 404 Not Found Screen. Displays a user-friendly error message when a requested page or resource is not found, handled by the Fault and Error Management Module (M9). 17

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is valuable in SC, where modifications are frequent, especially during initial development as the solution space is explored.

Our design follows the rules laid out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

3.1 Related Documents

This Module Guide should be read in conjunction with the following project documents:

- **Software Requirements Specification (SRS):** https://github.com/OCD-Rats-Capstone/OCD-Rat-Infrastructure/blob/main/docs/SRS-Volere/SRS.pdf
- **Module Interface Specification (MIS):** https://github.com/OCD-Rats-Capstone/OCD-Rat-Infrastructure/blob/main/docs/Design/SoftDetailedDes/MIS.pdf

- **Verification and Validation Plan:** https://github.com/OCD-Rats-Capstone/OCD-Rat-Infrastruc blob/main/docs/VnVPlan/VnVPlan.pdf
- **Hazard Analysis:** https://github.com/OCD-Rats-Capstone/OCD-Rat-Infrastructure/blob/main/docs/HazardAnalysis/HazardAnalysis.pdf

3.2 Document Organization

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section **??** specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

AC1: The layout, accessibility, and responsiveness of the front-end interface.

AC2: Fine-tuning or using a different query NLP Model.

AC3: The constraints of filter and query metadata field inputs.

AC4: The framework used for creating visualizations from data and presenting them.

AC5: The algorithms implemented for behavioural analysis or classification.

AC6: The virtual hosting location of Dr. Szechtman's dataset.

AC7: The implementation of the user authentication and access control mechanism for scalability.

AC8: The implementation of error detection and fault recovery mechanisms.

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: The read-only access to Dr. Szechtman's dataset.

UC2: The dataset hosted on FRDR being our primary and individual data source.

UC3: Scope of data processing analysis (Does not generalize rat behaviour trial data to other animal models or domains).

UC4: The software application's MVC design pattern.

UC5: Input/Output devices (Input: File and/or Keyboard, Output: File, Memory, and/or Screen).

UC6: User's computing device (The assumption that the user's device will support modern browsers with Javascript enabled, will not change)

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 2. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M1: Hardware-Hiding Module

M2: NLP Query Processor Module

M3: Front-End Interface Module

M4: API Layer Module

M5: Data Schema and Storage Module

M6: Data Visualization Module

M7: Data Processing Pipeline Module

M8: Authentication and Access Control Module (Descoped)

M9: Fault and Error Management Module

Level 1	Level 2
Hardware-Hiding Module	
Behaviour-Hiding Module	Front-End Interface Module API Layer Module Data Schema and Storage Module Data Visualization Module
Software Decision Module	NLP Query Processor Module Data Processing Pipeline Module Authentication and Access Control Module Fault and Error Management Module

Table 2: Module Hierarchy

6 Connection Between Requirements and Design

This section summarizes how the system design satisfies the requirements defined in the Software Requirements Specification (SRS). The traceability matrices in section 8 provide the detailed mappings. We highlight the main design rationale.

The functional requirements (SRS §6.1, FUNC.R.1–FUNC.R.7) are met through the coordinated behaviour of the M3, M4, M2, M5, and M7 modules, which together support querying, dataset retrieval, processing, and visualization. Non-functional requirements related to usability, appearance, and accessibility (SRS §6.2, APP.R.1–ACC.R.1) are addressed primarily by the M3 and M2 modules, which enforce a simple interface and natural-language interaction.

Performance, reliability, and scalability needs (SRS §6.2, SAL.R.1–SOE.R.2) are handled by backend modules such as M4, M5, M7, and M9, while security and maintainability (SRS §6.2, INT.R.1–IMM.R.2) are supported by M9. These design choices isolate complexity and align with information-hiding principles.

Anticipated change requirements map to the modules identified in Table 4, demonstrating that the system decomposition localizes change and supports future extensibility. For the complete requirements definitions, refer to the SRS document.

7 Module Decomposition

The system is decomposed according to principles of information hiding (Parnas et al., 1984). Each module is defined by its *Secrets* (hidden design decisions), *Services* (exposed functionality), and the software or technology used for implementation. Only leaf modules in the hierarchy (section 5) are directly implemented.

7.1 Hardware-Hiding Module (M1)

Secrets: Configuration of computing infrastructure including backend servers, databases, and frontend deployment environments.

Services: Provides physical and virtual infrastructure for computation, storage, and web services.

Implemented By: OS / Cloud Infrastructure

7.2 Behaviour-Hiding Modules

7.2.1 Front-End Interface Module (M3)

Secrets: Frontend state management, routing, and layout design.

Services: User interface for dataset search, visualization, filtering, and download.

Implemented By: React

7.2.2 API Layer Module (M4)

Secrets: API routing, endpoint definition, query optimization.

Services: Provides REST API for structured and natural language queries.

Implemented By: FastAPI, Python

7.2.3 NLP Query Processor Module (M2)

Secrets: Query interpretation logic and integration with language model APIs.

Services: Converts natural language queries into structured database commands.

Implemented By: Custom Python code

7.2.4 Authentication and Access Control Module (M8)

Secrets: User authentication and role-based access control.

Services: Manages secure access to data and APIs. **Note: This module was descoped from the final revision due to time constraints.**

Implemented By: FastAPI Security Layer (Descoped)

7.3 Software Decision Modules

7.3.1 Data Processing Pipeline Module (M7)

Secrets: Data processing algorithms and pipeline configuration for efficiency and scalability.

Services: Processes behavioral datasets to compute metrics and prepare data for visualization.

Implemented By: Python (NumPy, Pandas)

7.3.2 Data Schema and Storage Module (M5)

Secrets: Database schema, indexing strategies, and query optimization.

Services: Efficient storage and retrieval of behavioral datasets and metadata.

Implemented By: PostgreSQL

7.3.3 Data Visualization Module (M6)

Secrets: Visualization design and interactive plotting logic.

Services: Generates charts, trajectories, and heatmaps for research analysis.

Implemented By: React + Charting Libraries

7.3.4 Fault and Error Management Module (M9)

Secrets: Error detection, logging, and recovery mechanisms.

Services: Ensures robustness and reliability during runtime exceptions or unexpected input.

Implemented By: Python logging / monitoring frameworks

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Table 3: Trace Between Requirements and Modules

Requirements	Modules
FUNC.R.1	M3, M5, M7
FUNC.R.2	M2, M4, M5
FUNC.R.3	M3, M5, M7

Continued on next page

Table 3 (continued)

Requirements	Modules
FUNC.R.4	M5, M6, M7
FUNC.R.5	M3, M4, M6
FUNC.R.6	M3, M4, M5
FUNC.R.7	M1, M3, M4
APP.R.1	M3, M7
APP.R.2	M3
APP.R.3	M3, M7
EOU.R.1	M3, M2
EOU.R.2	M2, M3, M6
LEA.R.1	M2, M3
LEA.R.2	M3, M6
UAP.R.1	M2, M3, M4
UAP.R.2	M3
ACC.R.1	M3
SAL.R.1	M7
SAL.R.2	M4
POA.R.1	M5, M7
POA.R.2	M4, M5, M7, M9
ROFT.R.1	M3, M9
ROFT.R.2	M9
CAP.R.1	M4, M5
CAP.R.2	M1, M5
SOE.R.1	M4
SOE.R.2	M4, M7
LON.R.1	M9
LON.R.2	M4
EPE.R.1	M1, M4
EPE.R.2	M1, M4
EPE.R.3	M1
WE.R.1	M1, M3, M4
IWAS.R.1	M4, M5
PROD.R.1	No Module
PROD.R.2	No Module
REL.R.1	No Module

Continued on next page

Table 3 (continued)

Requirements	Modules
REL.R.2	No Module
MAI.R.1	M9
MAI.R.2	M4, M5
SUP.R.1	M3
SUP.R.2	M3
ADA.R.1	M1, M4
ADA.R.2	M1, M3, M4
INT.R.1	M9
IMM.R.1	M9
IMM.R.2	M4, M9
LEG.R.1	No Module
STA.R.1	No Module
STA.R.2	No Module
WR.1	M6, M7
WR.2	M5, M7
WR.3	M7

AC	Modules
AC1	M3
AC2	M2
AC3	M4, M5
AC4	M6
AC5	M7
AC6	M1, M5
AC7	M8
AC8	M9

Table 4: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Parnas (1978) said of two programs A and B that A *uses* B if correct execution of B may be necessary for A to complete the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of

B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

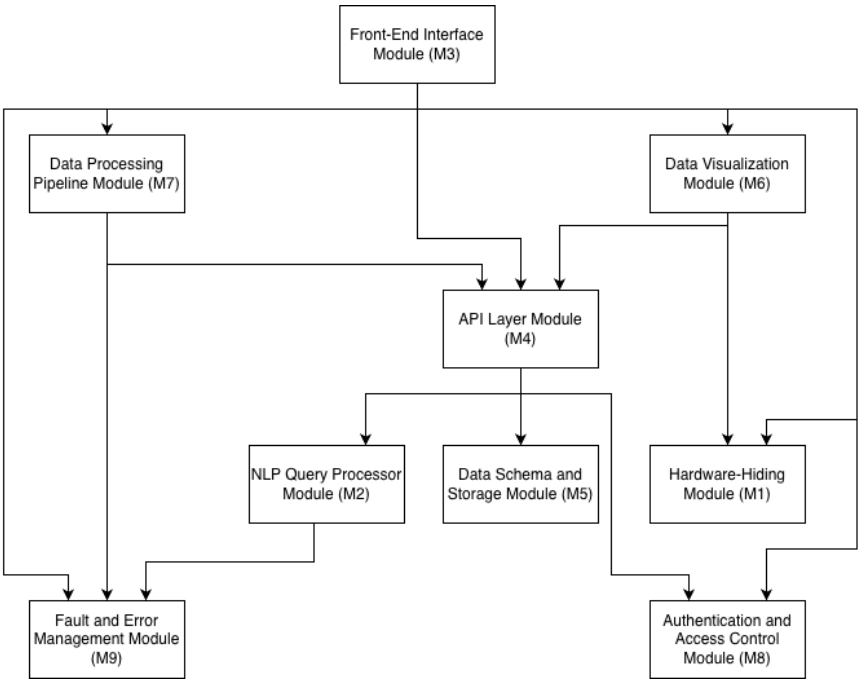

Figure 1: Use hierarchy among modules

10 User Interfaces

This section presents the main user interface designs for **InfraRat**, illustrating the structure and layout of the system's primary pages. The following figures show a revised user interface design of the application, developed to guide implementation and maintain a consistent user experience across the program. Each screenshot corresponds to functionality exposed by the Front-End Interface Module (M3) and related modules.

10.1 Homepage

The homepage serves as the primary landing page for the application and provides an overview of the system's purpose and capabilities. It introduces the platform to new users and provides navigation to the core features of the system, including the query interface and data visualization tools.

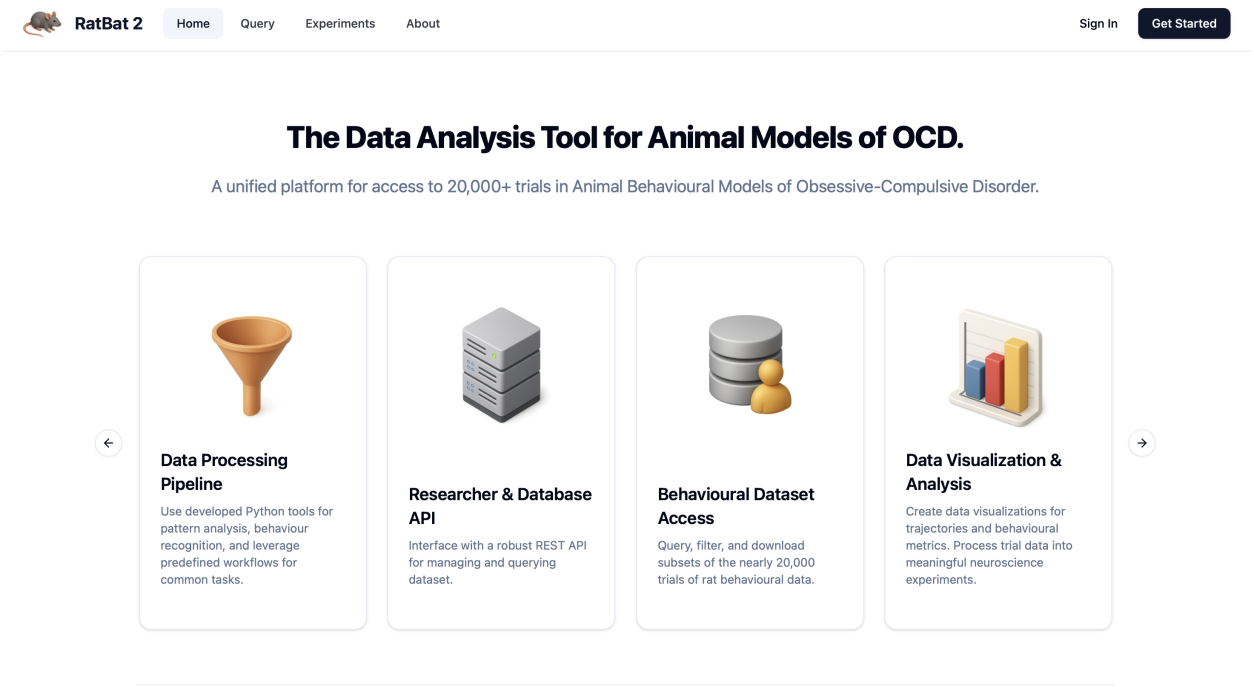

Figure 2: InfraRat Homepage – Initial Layout Frame 1. Displays the main navigation bar, branding, and hero section introducing the platform and its purpose to researchers.

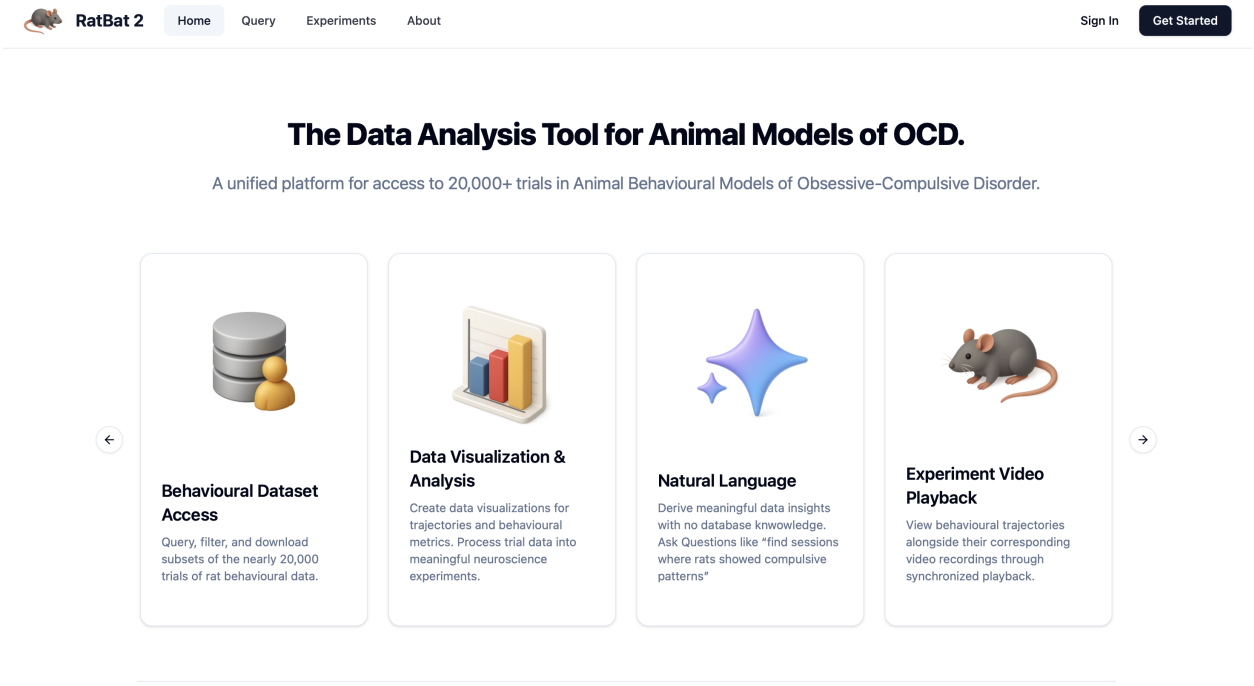

Figure 3: InfraRat Homepage – Initial Layout Frame 2. Continues the landing page content, highlighting key features of the platform such as dataset querying and visualization capabilities.

The subsequent homepage frames demonstrate the trial data overview and natural language processing information sections, which give users a preview of the data available through the system (M5) and the NLP-powered search capability (M2).

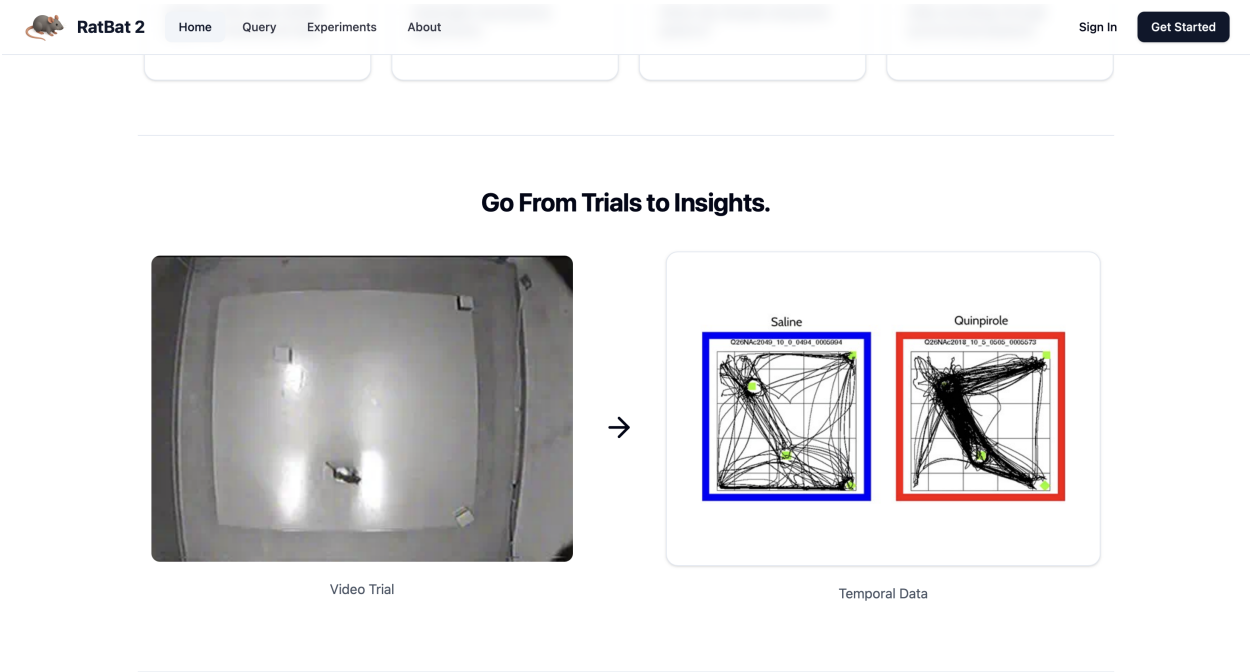

Figure 4: InfraRat Homepage – Trial Data Display. Shows a summary view of trial data records retrieved from the Data Schema and Storage Module (M5), giving users a preview of the structured experimental session data.

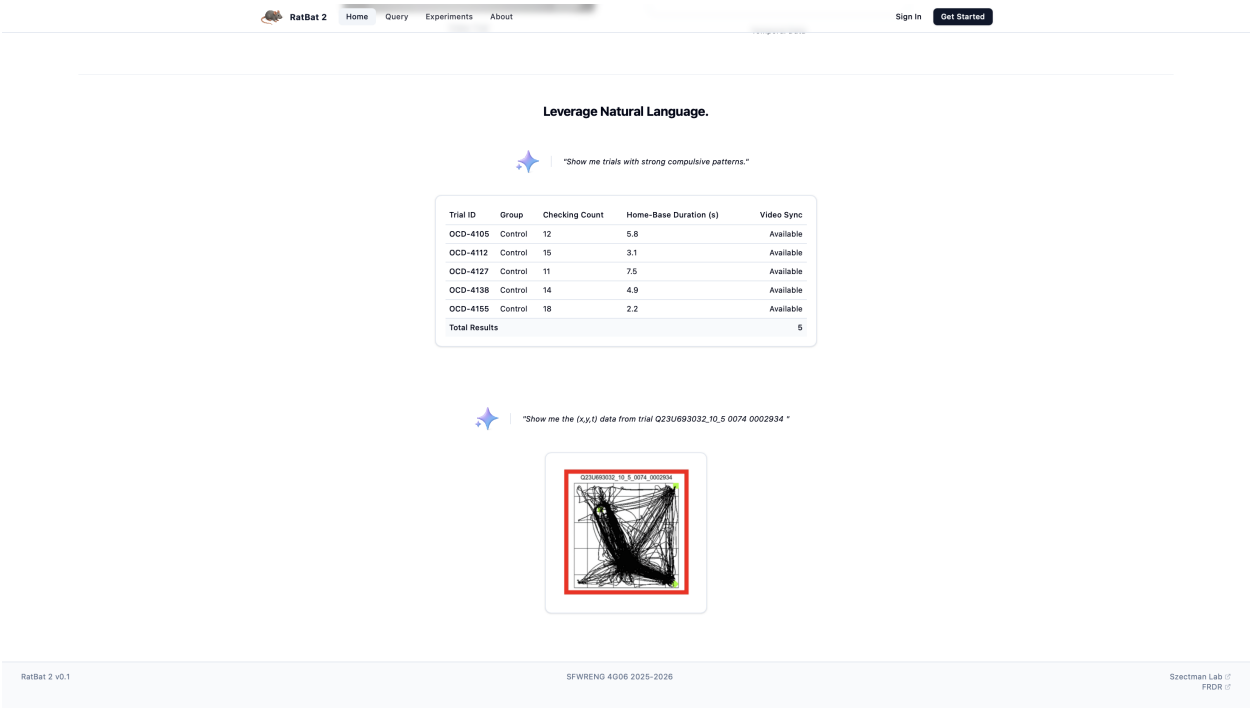

Figure 5: InfraRat Homepage – NLP Information. Presents information about the natural language query capability powered by the NLP Query Processor Module (M2), explaining how researchers can use conversational input to search the dataset.

10.2 Query Page

The query page is the primary interaction point for dataset retrieval. It provides both structured filter-based search and natural language input and displays results returned from the API Layer Module (M4).

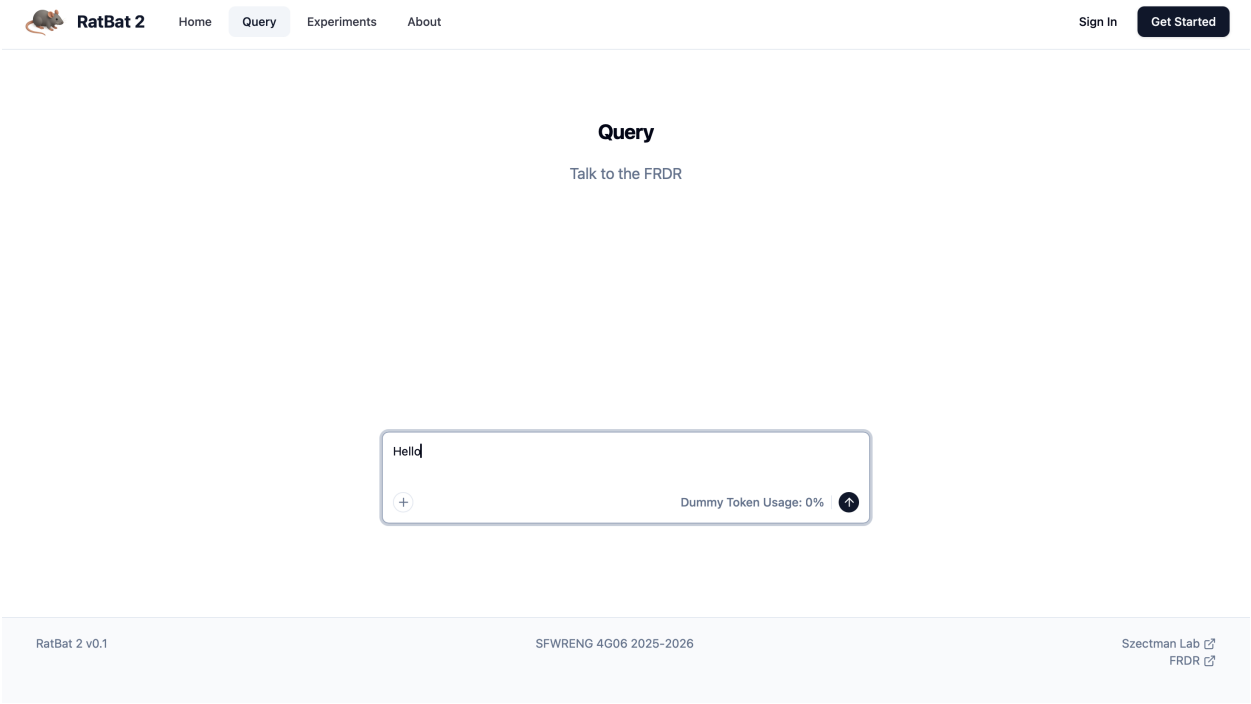

Figure 6: Query Page – Search and Filter Interface. Displays the dual-mode query interface where users can either apply structured metadata filters (left panel) or enter natural language queries (central input). This page exercises the Front-End Interface Module (M3), API Layer Module (M4), and NLP Query Processor Module (M2).

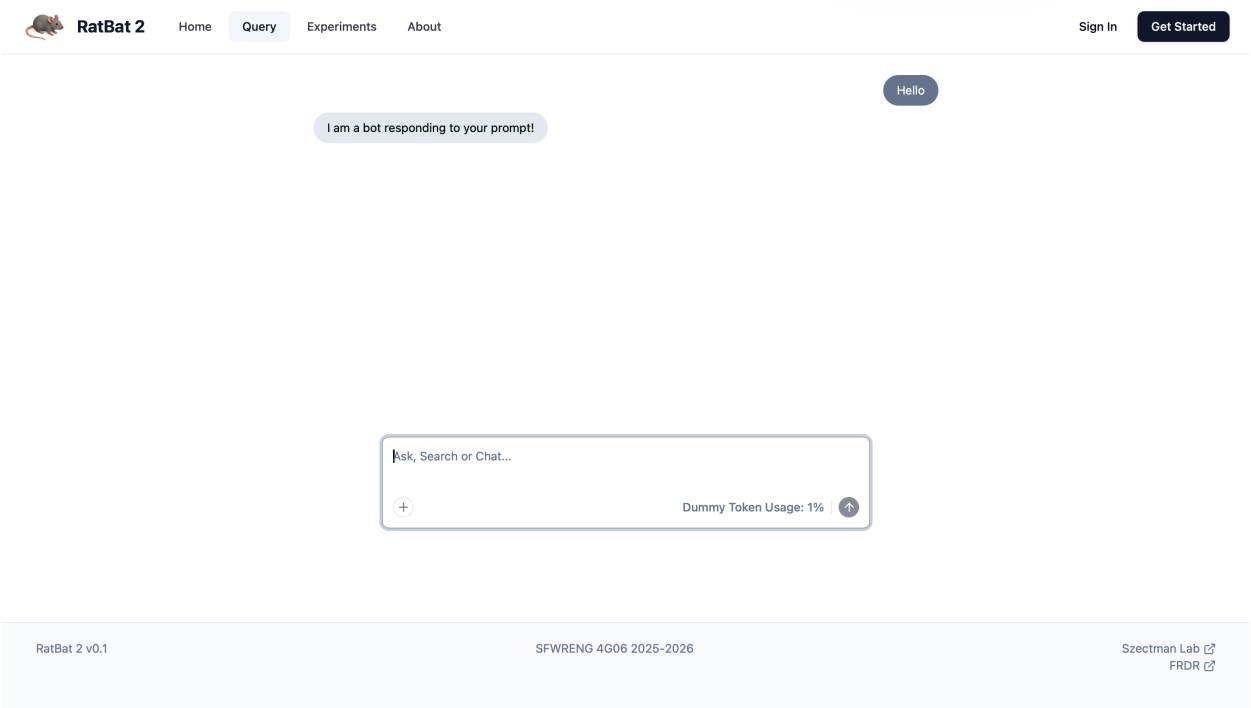

Figure 7: Query Page – Test of Sample Prompt. Demonstrates a completed query with results displayed in a tabular format returned from the Data Schema and Storage Module (M5) via the API Layer (M4). Users can view, sort, and download the returned dataset records.

10.3 Experiment and About Pages

The experiment page provides an overview of experimental configurations and management, while the about pages present team information and background context for the platform.

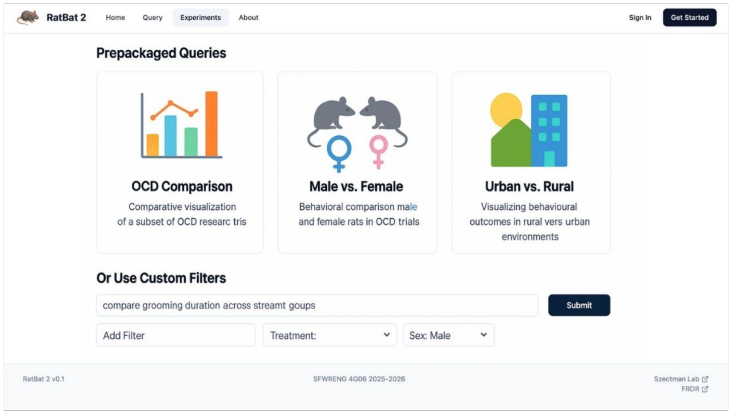

Figure 8: Experiment Page – Experiment Management and Overview. Allows users to browse and manage experimental groupings, linking sessions through the Data Schema and Storage Module (M5).

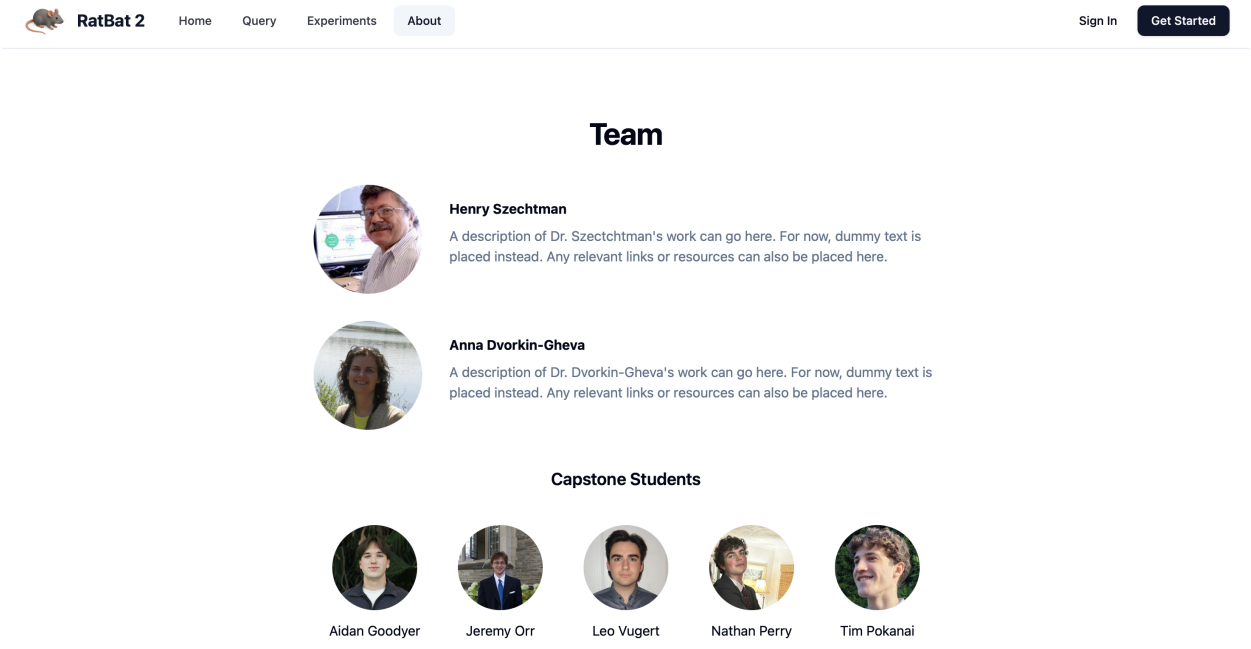

Figure 9: About Page – Team Section. Presents team member information and the project's academic context.

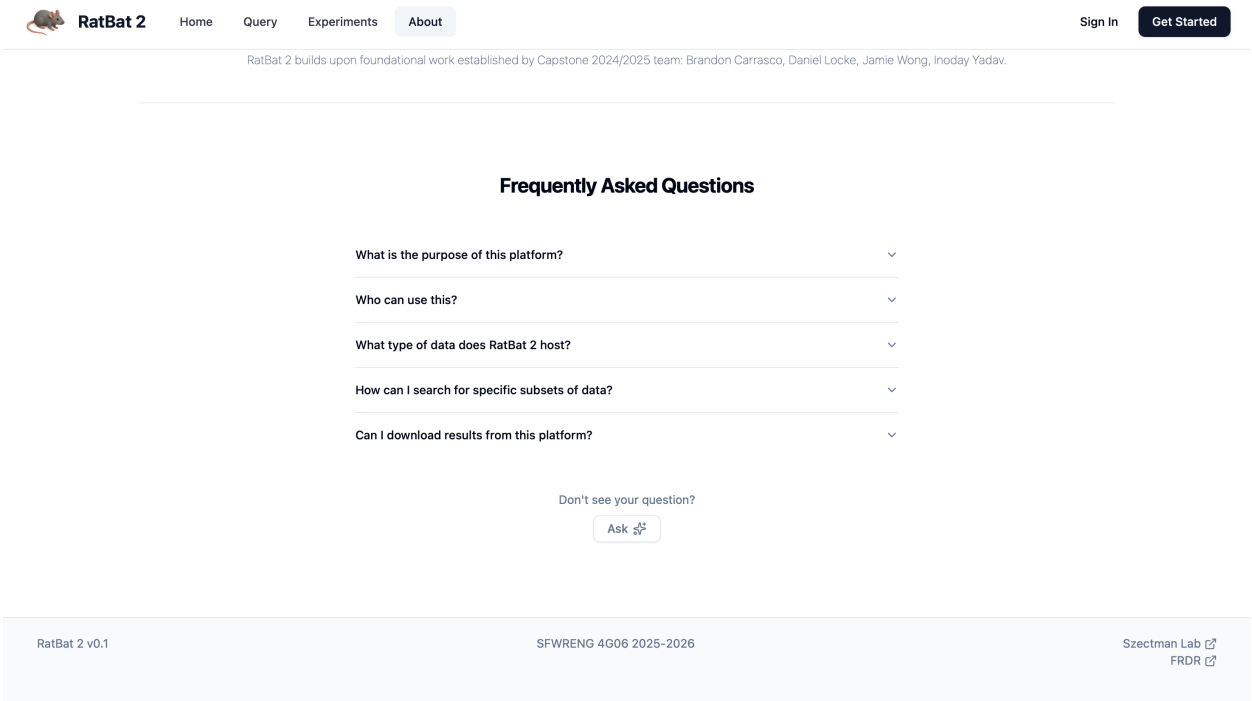

Figure 10: About Page – Frequently Asked Questions (FAQ) and Background Information. Provides users with contextual information about OCD research, the FRDR dataset, and instructions for using the platform.

10.4 Error Handling

The error page demonstrates the user-facing error handling provided by the Fault and Error Management Module (M9) in coordination with the Front-End Interface Module (M3).

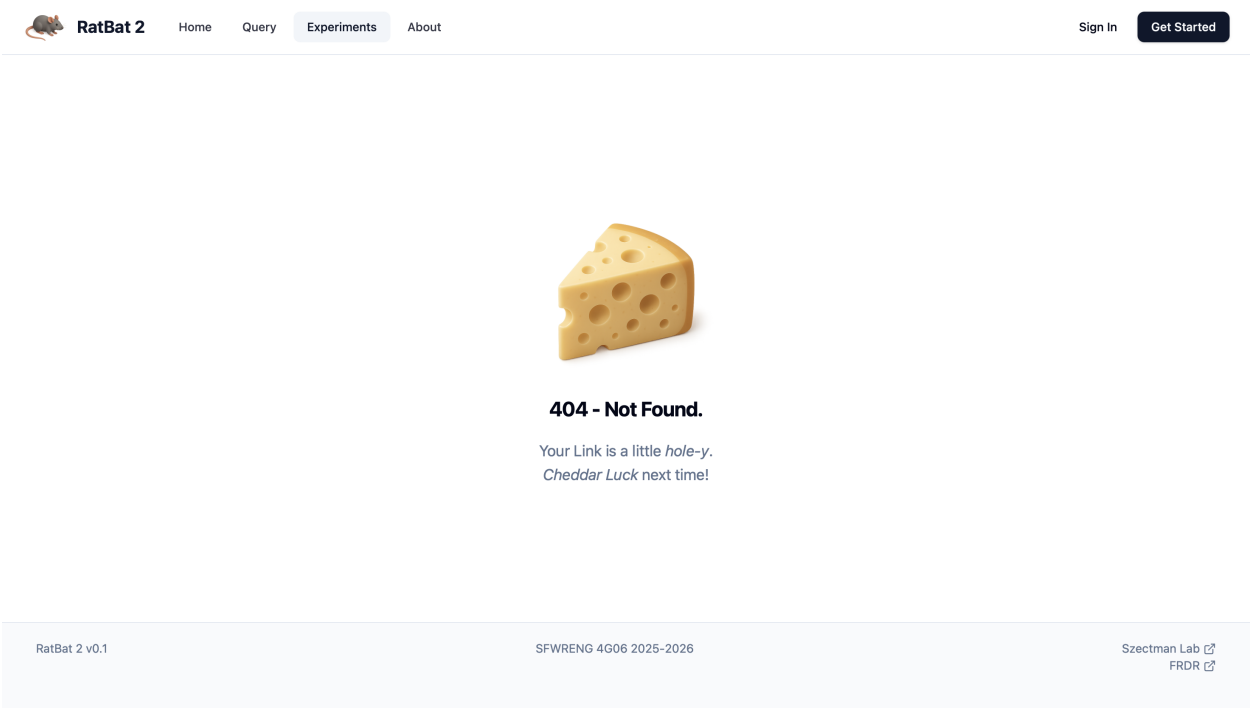

Figure 11: Error Page – 404 Not Found Screen. Displays a user-friendly error message when a requested page or resource is not found, handled by the Fault and Error Management Module (M9).

11 Design of Communication Protocols

The communication protocols define how system modules exchange information and coordinate data flow between the user interface, processing components, and data sources. The goal is to maintain consistent, secure, and modular interactions across all subsystems.

11.1 Overall Communication Architecture

All communication follows a client–server model where the Front-End Interface Module (M3) acts as the client and the API Layer Module (M4) serves as the central hub between client requests and backend modules. Data is exchanged in JSON format over HTTPS to ensure interoperability and security.

11.2 External Communication

The front-end sends RESTful HTTP(S) requests to the API layer for querying behavioural datasets, submitting NLP-based searches, and requesting visualizations. Responses are returned as JSON objects containing results, metadata, or error information.

11.3 Internal Communication

The API Layer (M4) intermediates between modules:

- Sends parsed queries to the NLP Query Processor (M2).
- Retrieves structured data from the Data Schema and Storage Module (M5).
- Forwards subsets to the Data Processing Pipeline (M7) for analysis.
- Provides outputs to the Data Visualization Module (M6).
- Coordinates with Authentication and Access Control (M8) for validation.
- Works with Fault and Error Management (M9) for monitoring and recovery.

11.4 Communication Reliability and Standards

All communication uses HTTPS/TLS encryption. Backend interactions are stateless, enabling independent module deployment. Standard HTTP response codes are used for consistency (e.g., 200, 400, 401, 500). Asynchronous queues handle long-running tasks to prevent blocking.

11.5 Scalability and Fault Tolerance

Modules support horizontal scaling through containerization. The Fault and Error Management Module (M9) provides retry and fallback mechanisms. The Hardware-Hiding Module (M1) abstracts hosting details, allowing migration across environments without affecting communication.

12 Timeline

The project timeline outlines the major milestones, deliverables, and responsibilities for each development phase at the module level. The table below provides a detailed breakdown of tasks, individual assignees, and actual completion dates. Issue tracking and supplementary progress details are maintained publicly through the project's GitHub repository at https://github.com/OCD-Rats-Capstone/OCD-Rat-Infrastructure/issues.

The Rev 0 implementation period spanned from January 14 to February 11, 2026. Subsequent revisions continued through March 2026 with additional visualization, data processing, and testing work.

Table 5: Detailed Project Timeline and Responsibilities

Module	Task	Assignee(s)	Start	End	Status
Phase 1: Foundation and Infrastructure (Jan 10–27)					

Table 5 (continued)

Module	Task	Assignee(s)	Start	End	Status
M1	Dockerize frontend and backend services	Aidan Goodyer	Jan 10	Jan 19	Done
M1	CI/CD pipeline: self-hosted runner, deploy workflow	Aidan Goodyer	Jan 23	Jan 27	Done
M1	Staging environment with port-based separation	Aidan Goodyer	Jan 25	Jan 27	Done
M5	Database schema development (with Dr. Szechtman)	Nathan Perry	Pre-Jan	Jan 14	Done
Phase 2: Core Feature Development (Jan 14–Feb 6)					
M2	Build NLP-to-SQL interpreter with LLM API	Nathan Perry	Jan 23	Jan 28	Done
M2	Integrate local LLM prototype for dev environments	Aidan Goodyer	Jan 10	Jan 17	Done
M2	Refine LLM: expose query, SQL, and rationale to UI	Aidan Goodyer	Jan 13	Jan 17	Done
M3	Implement filter page with structured criteria	Jeremy Orr	Nov 21	Feb 2	Done
M3	Develop inventory-style attribute filter UI	Jeremy Orr	Jan 28	Feb 4	Done
M3	Navbar and general UI refinements	Tim Pokanai	Feb 4	Feb 5	Done
M4	Backend server and REST API endpoints	Nathan Perry	Nov 17	Jan 28	Done
M4	Connect structured queries to PostgreSQL	Nathan Perry	Jan 28	Feb 6	Done
M4	SQL injection hardening and CodeQL fixes	Aidan Goodyer	Feb 4	Feb 4	Done
M4	File download from FRDR URLs	Nathan Perry	Jan 25	Feb 11	Done
Phase 3: Visualization and Processing (Jan 29–Mar 28)					
M6	Initial data visualization (bar charts, front-end)	Leo Vugert	Nov 26	Feb 5	Done

Table 5 (continued)

Module	Task	Assignee(s)	Start	End	Status
M6	Heatmap visualization implementation	Tim Pokanai	Feb 4	Feb 4	Done
M6	Line chart and time-series visualizations	Tim Pokanai	Feb 4	Mar 26	Done
M6	Spatial heatmap for positional data	Leo Vugert	Mar 26	Mar 28	Done
M6	Velocity profile visualization	Tim Pokanai	Mar 29	Mar 29	Done
M7	Path plot trajectory rendering	Nathan Perry	Mar 5	Mar 11	Done
M7	Summary distance metrics and analysis toolbox	Nathan Perry	Mar 3	Mar 26	Done
M7	Database with summary measures integration	Nathan Perry	Mar 17	Mar 17	Done
M6	Dr. Szechtman's specific analysis queries (10a)	Tim Pokanai	Mar 26	Mar 28	Done
M6	Standard error of mean in bar charts	Tim Pokanai	Mar 26	Mar 26	Done
Phase 4: Testing and Validation (Mar 5–Apr 5)					
M9	Backend unit test suite	Aidan Goodyer, Nathan Perry	Mar 5	Mar 9	Done
M9	Automated testing (VnV Report)	Tim Pokanai	Mar 9	Mar 9	Done
M3	UI refinements post-user testing	Aidan Goodyer	Mar 26	Mar 26	Done
M8	Authentication and Access Control	N/A	N/A	N/A	Descoped (Time Constraints)

Ongoing updates, issue tracking, and future work continue to be maintained through GitHub to ensure transparency and collaboration with supervisors and contributors.

References

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

David L. Parnas. Designing software for ease of extension and contraction. In *ICSE '78:*

Proceedings of the 3rd international conference on Software engineering, pages 264–277, Piscataway, NJ, USA, 1978. IEEE Press. ISBN none.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.